



Application Note

Next Generation of User Interface (UI) Controls and Real Script Functions in the Vantage™ System Programming Model

Dr. Daniel Rohrbach
Sr. Software Engineer and
Scientist
Verasonics Inc.

September 2020

Summary

With 4.3.0 release, users are now able to use real function definitions as callbacks in external functions and UI-controls in their scripts. Although backward compatible, this new scheme replaces the former text2cell approach, which allowed users to specify code for their callback functions within a single script. Defining actual functions in a script was not possible due to limitations of older Matlab versions (<R2016b). We believe that defining real functions in a script will provide users a more Matlab-natural way of defining callbacks and additionally provides many benefits. For example, users can now debug and set breakpoints directly in a script. Since Matlab recognizes the new functions, there is also improved syntax highlighting and error checking. Furthermore, these changes restore full compatibility with Matlab releases 2019b and later, for embedded functions declaring persistent variables.

In order to facilitate the new approach, we developed a modern object-oriented design for defining user UI-controls. When designing these new features, it was important to maintain the flexibility and feel of defining sequences that many of our customers value. However, we also wanted to add new features to make script development easier. Therefore, we designed the new objects so that they behave very similarly to the former structs, but now provide instant Matlab-formatted help and parameter-error checking directly from within the script or command prompt. Furthermore, we hope the new software designs will make it easier to provide more advanced and upgraded features to our customers. For example, with release 4.3.0 you can now define a dropdown menu in addition to the button group.

Table of Contents

Summary	2
Table of Contents	2
Document Change History	3
1. Introduction and Motivation	3
1.1 Background	3
1.2 The original approach	4
1.3 Limitations of the old approach	5
2. The new approach	7
2.1 Comparison of old and new Approach	7
2.2 Benefits of the new approach	8
3. How to convert a script to the new approach	11
3.1 Example scripts that demonstrate the new approach	11
3.2 Step by step guide for converting external functions (EF)	11
3.3 Step by step guide UIC	13

4.	Advanced programming and detailed usage	15
4.1	How to get started: The Package structure.....	15
4.2	Getting instant help.....	16
4.3	Usage	17
4.3.a	<i>Easy approach</i>	17
4.3.b	<i>Use the original definition to create new objects</i>	18
4.3.c	<i>Use value-parameter input</i>	19
4.4	Properties.....	19
4.5	Instant Error Checking	20
4.6	<i>Differences to the original approach</i>	21
4.7	The new Callback function handles	22

23

Document Change History

Date	Section(s)	Description	Author
July 23, 2020	All	Rev A: Initial document	DR
September 4, 2020	Table of Content	Rev B: updated table of content to match document	DR

1. Introduction and Motivation

1.1 Background

An integral part of the Verasonics programming model enables researcher's flexibility to develop and execute custom user-written code in real time. This key mechanism allows users to generate virtually any ultrasound application that can be considered for research and development in ultrasonics.

Verasonics software provides two pivotal mechanisms to accomplish this flexibility, which are external-functions (EF) and UI control (UIC) callbacks. External functions allow access to a variety of data-buffers and execution operations on data at any point of the acquisition sequence. These mechanisms are ideal for testing functions such as custom ultrasound processing algorithms, new reconstruction routines, or novel imaging filters. The UI-Controls allow definition of dynamic interactions and controls using the Research GUI. This feature delivers the versatility and flexibility to test different parameter combinations in real-time.

Verasonics provides various example scripts that demonstrate these features. From an educational standpoint, it was important for us, that we are able to present each of these scripts as a self-consistent document. This meant that example code for EF and UIC should be included in the scripts. However, when Verasonics developed the first example scripts, we would not have been able to provide this feature to our customers because at that time, it was not allowed to define actual functions in scripts. This feature was added much later by Mathworks with version 2016b and later. Therefore, Verasonics developed a procedure that was termed the text2cell approach. That approach involves putting a block of code at the end of the script marked with enclosing keyword delimiters, then use a function called `text2cell` to reference this block of code as the callback in an EF or UIC.

Mathworks with Matlab version 2019b is further improving security and functionality of nested functions within scripts, which also bans persistent variables inside the code-body of a script. That means that scripts will now report an error if the user use the persistent key word in Matlab R2019b and newer, even if the code is not executed. This is the case for some of our advanced example scripts, where some callbacks define persistent variables.

Verasonics is determined to provide the most up to date technologies to customers, which includes supporting the latest MATLAB® versions. Therefore, our team wanted to find a solution to the R2019b issue, which not only fixes the issue with the persistent key word but also provides a familiar interface to our users and at the same time includes additional capabilities and benefits for programming of their Vantage system.

1.2 The original approach

To facilitate creating user controls on the Verasonics GUI, the Vantage software provides some built-in UI functions that can be positioned at certain predefined locations.

The current available controls that can be selected are: `VsSliderControl`, `VsPushButton`, `VsToggleButton`, `VsDropDown`, and `VsButtonGroup`.

Originally, defining UIC consisted of defining a struct with up to three elements

- `UI.Control` - defining the UIC object properties
- `UI.Callback` - a callback function executed when the state of the UIC is changing
- `UI.Statement` - some code that will be executed before the UIC is created

External functions (EF) are defined in a process structure (please refer to the Vantage programming manual for more details). The process structure defines the name of the method, and input and out parameter, which are usually referring to data buffers:

```
% Process structure for calling the external processing function
Process(2).classname = 'External';
Process(2).method = 'runTimeMon';
Process(2).Parameters = {'srcbuffer', 'none', ... % name of buffer to process.
                        'dstbuffer', 'none'};
```

To assign a callback to UICs and EFs the text2cell function was utilized. This function parses a block of code that is found in-between a set of delimiters. The delimiter key word is passed as an argument and usually is a MATLAB commented string (e.g., '%myDelimiter'). Text2cell then searches the code of the current executing script to find the block of code that is between the defined delimiter and stores each line of this block of code in a cell array. Please see the example below for more details.

When the script data is loaded into VSX, the main program then converts these lines of code into an actual Matlab function and saves it as an m-file in a temporary folder during runtime.

Example:

```
% Define the UI control
UI(2).Control = {'UserA1','Style','VsSlider','Label',['Range (','AxesUnit,')'],...
% assign the callback
UI(2).Callback = text2cell('%-UI#2Callback');
```

The callback would then be defined at the end of your script encompassed between the delimiter '%-UI#2Callback'.

```
%-UI#2Callback
...
Some call back code
...
%-UI#2Callback'
```

External functions are defined similar

```
% assign the callback
EF(1).Function = text2cell('%-EFCallback_1');
```

With code defined at the end of the script

```
%-EFCallback_1
...
Some call back code
...
%-EFCallback_1
```

1.3 Limitations of the old approach

Although the original text2cell approach is easy to understand and read, it has some significant limitations:

- **Error Checking:** While constructing the UI control in a script, the user will not know whether the parameters that were defined are valid. Possible errors are: Misspelled property names, invalid parameter values (e.g., invalid range), or undefined parameter names due to misspelled field names.
- **Persistent Variables:** From Matlab version 2019b onwards, persistent variables are not allowed in scripts anymore even if that code section is not getting executed. In our original

approach, callback functions were defined as text sections in the script surrounded by user defined delimiters, and thus appear as part of the script. This will trigger an error if the external function defines a persistent variable. The reason for this approach was that in previous Matlab versions, functions were not allowed in scripts. However, to make our scripts more self-contained, we believe that it is of advantage to have callback functions inside the scripts.

- **Function Debugging:** Callback functions cannot easily be debugged in the original approach. Debugging functions involved modifying the actual code and place a `keyboard` variable inside the callback routine. This will cause the execution to stop at that location and wait for user input. Although sufficient, this is not very practical because the function where the breakpoint will be hit is a temporary function that will be overwritten the next time VSX is executing (see section 1.1). This means that all changes to that function will be lost. Furthermore, this does not allow for dynamic placement of breakpoint as it is possible with real functions.
- **Help and support:** Since the UI-control definitions are pure structs, we cannot provide real help inside the Matlab environment. the help command will not provide any useful information and the only way of obtaining help is by consulting the Vantage Sequence-programming manual

2. The new approach

2.1 Comparison of old and new Approach

When designing the new features, it was critical to maintain the flexibility and user-familiar experience of defining a sequence using our scripting language. The intention is to make the transition to this new approach as easy as possible for users while maximizing the benefits of the improved capabilities. In fact, only a few modifications are required to switch from the original text2cell approach to the new UI-control scheme.

Old approach	New approach
<pre> 271 % - Sensitivity cutoff change callback 272 %SensCutoffCallback 273 ReconL = evalin('base', 'Recon'); 274 for i = 1:size(ReconL,2) 275 ReconL(i).senscutoff = UIValue; 276 end 277 278 ... code to assign the parameters to the system 279 280 return 281 %SensCutoffCallback 282 283 %RangeChangeCallback 284 285 ... code to obtain parameters 286 287 P.endDepth = UIValue; 288 Display = Resource.DisplayWindow(1); 289 290 291 if isfield(Display(1), 'AxesUnits') && ~isempty(Display.AxesUnits) 292 if strcmp(Display.AxesUnits, 'mm') 293 P.endDepth = UIValue*scaleToWvl; 294 end 295 end 296 297 ... code to assign the parameters to the system 298 299 %RangeChangeCallback </pre>	<pre> 356 % - Sensitivity cutoff change callback 357 function SensCutoffCallback(hObject, event, UIValue) 358 ReconL = evalin('base', 'Recon'); 359 for i = 1:size(ReconL,2) 360 ReconL(i).senscutoff = UIValue; 361 end 362 363 ... code to assign the parameters to the system 364 365 end 366 367 % - Range change callback 368 function RangeChangeCallback(~, ~, UIValue) 369 370 ... code to obtain parameters 371 372 P.endDepth = UIValue; 373 Display = Resource.DisplayWindow(1); 374 375 376 if isfield(Display(1), 'AxesUnits') && ~isempty(Display.AxesUnits) 377 if strcmp(Display.AxesUnits, 'mm') 378 P.endDepth = UIValue*scaleToWvl; 379 end 380 end 381 382 ... code to assign the parameters to the system 383 384 end </pre>

Figure 1 Function definition in scripts, old vs new approach

Define actual MATLAB functions inside the script for any callbacks:

Figure 1 shows an example of a sensitivity and a range callback and compares the old and new scheme. Before, in the text2cell-approach, users defined a callback using a set of encompassing keyword delimiters (e.g., SensCutoffCallback, RangeChangeCallback). In the new approach users can define an actual MATLAB function using the function keyword (e.g., function SensCutoffCallback, function RangeChangeCallback).

Use real Objects to define the UI-Controls:

The new function definitions can be used directly as function handles (i.e., using the @ operator) in external functions or UI-control callbacks. In order to facilitate this new approach, Verasonics developed a modern object-oriented (OO) design for defining user UI-controls, which together with the new function definitions will replace the old text2cell-method

This might sound complicated at first, however, we tried to keep it as simple as and as close as possible to the original scheme. Furthermore, it is not required to learn OO programming. Verasonics introduced a few new functions, which can be conveniently copied from the numerous

example scripts. In fact, some of the benefits will make it actually easier to work with this new scheme. Figure 2 shows two examples of UI-control definitions and an example of the External function definition comparing between old and new approach.

Old approach	New approach
UI-control definition	
<pre> 216 % - Sensitivity Cutoff 217 UI(1).Control = {'UserB7','Style','VsSlider', ... 218 'Label','Sens. Cutoff',... 219 'SliderMinMaxVal',[0,1.0,Recon(1).senscutoff], ... 220 'SliderStep',[0.025,0.1], ... 221 'ValueFormat','%1.3f'}; 222 UI(1).Callback = text2cell('%SensCutoffCallback'); 223 224 225 UI(2).Control = {'UserA1','Style','VsSlider', ... 226 'Label','Range (',AxesUnit,')',... 227 'SliderMinMaxVal',MinMaxVal, ... 228 'SliderStep',[0.1,0.2], ... 229 'ValueFormat','%3.0f'}; 230 UI(2).Callback = text2cell('%RangeChangeCallback'); 231 </pre>	<pre> 267 % - Sensitivity Cutoff 268 UI(1).Control = VsSliderControl('LocationCode','UserB7',... 269 'Label','Sens. Cutoff',... 270 'SliderMinMaxVal',[0,1.0,Recon(1).senscutoff],... 271 'SliderStep',[0.025,0.1], ... 272 'ValueFormat','%1.3f', ... 273 'Callback', @SensCutoffCallback); 274 275 276 UI(2).Control = VsSliderControl('LocationCode','UserA1', ... 277 'Label','Range (',AxesUnit,')',... 278 'SliderMinMaxVal',MinMaxVal, ... 279 'SliderStep',[0.1,0.2], ... 280 'ValueFormat','%3.0f', ... 281 'Callback', @RangeChangeCallback); 282 </pre>
External function definition	
<pre> 243 % Create the External Processing function, using original text2cell 244 EF(1).Function = text2cell('%EF#1'); 245 246 </pre>	<pre> 296 % Create the External Processing function, using new approach 297 EF(1).Function = ExFunctionDef('runTimeMon', @runTimeMon); 298 299 </pre>

Figure 2 Comparison of UI-control definition and external function definition.

2.2 Benefits of the new approach

Script Readability:

Defining real functions in a script will provide a more Matlab-natural and familiar way of defining callbacks (compare old vs. new approach in Figure 1). Users will experience scripts with a cleaner look, and it will be easier to find and distinguish different callback functions. Additionally, the new approach provides many benefits.

Debugging:

For example, users can now debug and set breakpoints directly in their script (Figure 3). With the new approach users do not have to insert any code such as keyboard to indicate a location where they want to interrupt the processing of their script. In fact, users can now even set breakpoints dynamically while running scripts without the need of restarting VSX. All changes to the code will be applied directly to the script and not to the temporary created function.

Syntax highlighting:

Furthermore, our users will also see improved syntax highlighting and error checking for callbacks as MATLAB now recognizes the new functions. See Figure 1, for example, MATLAB identifies that hObject, and event, in SensCutoffCallback function are not used and reports a warning. Another benefit of the changes is that it also restores full compatibility with MATLAB releases 2019b and later, for embedded functions, declaring persistent variables and other keywords that were not allowed in scripts previously.

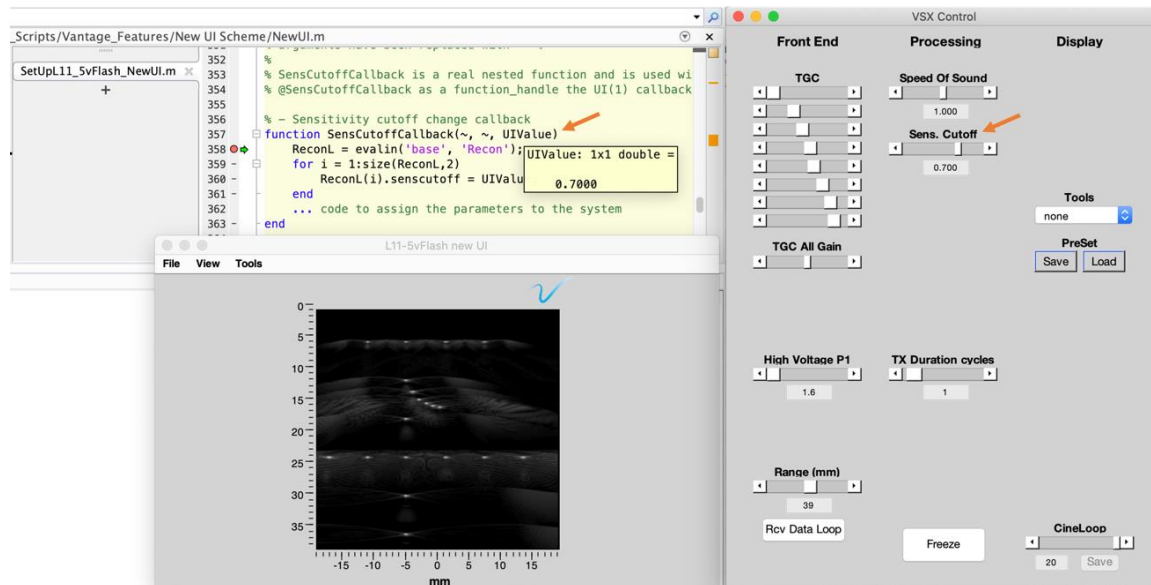


Figure 3 Debugging your script directly, after setting a breakpoint directly in your script, you can observe changes to GUI values (orange arrows)

Integrated help function:

Additional benefits come with the new OO approach for defining the UI-controls and external functions (Figure 2). With the definition of the objects we integrated the MATLAB help functions that ease the use of the tools. That means, it is not necessary to look up the definition of a `VsSliderControl` in the users programming manual. Now, users can simply utilize the MATLAB integrated help function directly. The two features are demonstrated in Figure 4 and Figure 5. The new UI-control functions are defined in a package structure. The parent, top-level package is `vsv`. To find the `VsSliderControl` definition users can hit the TAB key on the keyboard after each `'.'` to get a suggestion of available functions for each package (Figure 4). This releases the burden of having to remember the exact function names every time they are needed.

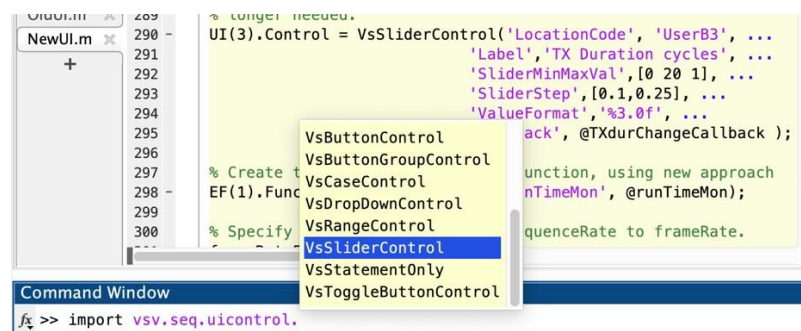


Figure 4 shows Matlab intelligence for code suggestions after `'.'` Operator

Users can utilize the import command to import entire packages to their current workspace. When `help` is typed followed by the function name, extensive help with code examples to each of the available functions will be printed to the MATLAB Command Window (Figure 5).

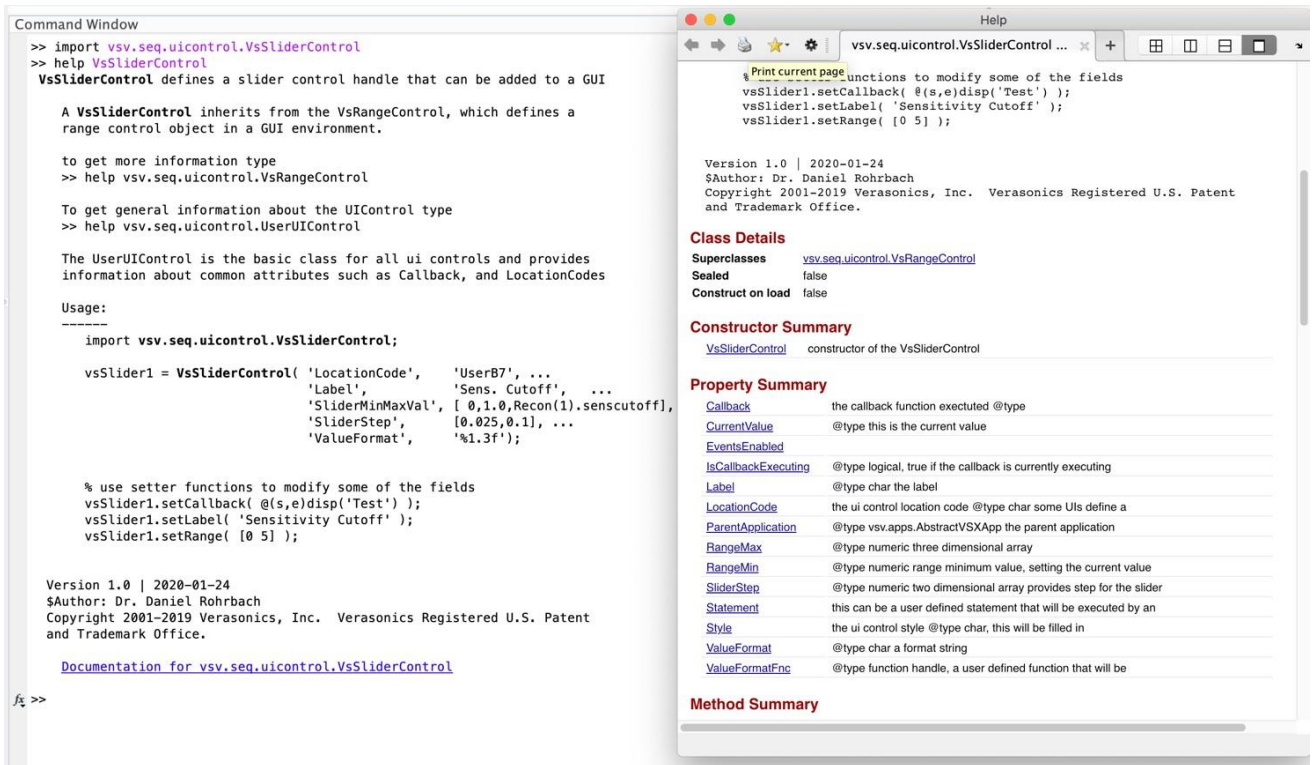


Figure 5 Using help for the `VsSliderControl` will print comprehensive help with usage examples. When clicking on the link at the end users will be able to review further details of the class in MATLAB's help window (right window). Further details about the properties (i.e., fields) and methods will be provided with a detailed description of the parameter types, default values and limits.

Error checking:

A novel benefit of the objects is that they can be used in the same way as the familiar structs while making scripts less error prone and more stable at the same time. For example, a common mistake is to accidentally misspell a parameter name. In the worst case, VSX would execute the script without reporting any error and the user may not even realize that the parameter value did not get applied. In the best case, VSX would throw an error. The new OO approach makes it impossible to misspell field names or provide invalid parameters (e.g., a char array instead of a double value) because the objects will check parameter validity directly when the script is executed (i.e., before running VSX). This new approach provides a more direct error reporting, and users do not need to run VSX first to find out whether a mistake is present.

3. How to convert a script to the new approach

Many of Verasonics users may wonder how easy it is to change the scripts to the new approach and whether all scripts are required to be modified in order to use them with the new software releases. The good news are: Unless a user defined a persistent variable in one of their callbacks and used MATLAB R2019b or higher, they will not need to change the scripts. Verasonics designed the new approach to be backwards compatible. In the current release, users will find examples that use the text2cell-approach and some that use the new approach. For now, the new approach was only used for all example scripts that use a persistent variable.

However, Verasonics recommends that users spend a few minutes to familiarize themselves with the new approach and update their scripts, in order to get access to the new features and benefits. The required modifications are rather simple. Only two modifications are required for this update. First, is converting the Callback code to actual functions and second is to use the new UI-Control definitions for defining sliders, buttons, button groups, and other tools. For convenience, Verasonics is providing two example scripts that show the existing and updated approaches together with comprehensive documentation and user manuals.

3.1 Example scripts that demonstrate the new approach

We provide two example scripts that compare the old and new approach. Furthermore, we converted all example scripts that defined a persistent variable in an EF or UIC to the new approach to make it compatible with the newest Matlab versions.

The easiest way to convert your scripts is to study the example scripts. The comparisons scripts can be found in

[Vantage-4.3.0/Example_Scripts/Vantage_Features/New UI Scheme](#)

The two new example scripts are derivatives of the original L11-5v flash scripts with an added EF to demonstrate the approach. The names of the two example scripts are

- [SetUpL11_5vFlash_OldUI](#)
- [SetUpL11_5vFlash_NewUI](#)

The script demonstrating the new approach has extensive documentation to guide the user to the new approach.

3.2 Step by step guide for converting external functions (EF)

- A. Find all Process objects with the class name saying 'External' and note the method name (e.g. 'ROIplot' in the example below)

```
% EF1 is external function for ROI plot
Process(4).classname = 'External';
Process(4).method = 'ROIplot';
```

```
Process(4).Parameters = {'srcbuffer', 'none'};
```

- B. Search your script for external function definition `EF`, note the delimiter (e.g., `'%-EF#1'`) specified as input to `text2cell`

```
% - External function for ROIplot
EF(1).Function = text2cell('%-EF#1');
```

- C. Find the text that is enclosed by the delimiter

```
%-EF#1
ROIplot(varargin)
persistent drawROI

% change the txt for voltage slider
hvltxt = findobj('tag','hvltxt');
hv2txt = findobj('tag','hv2txt');
set(hvltxt,'String','Bmode Voltage');
set(hv2txt,'String','Doppler Voltage');

% drawRegionOutline(WinNum,PDataNum,RegionNum) creates an outline from
% PData(PDataNum).Region(RegionNum) on displayWindow(WinNum) with default color -
white.
if isempty(drawROI)
    drawROI = 1;
    evalin('base','hROI = drawRegionOutline(1,2,numRaysDop+1);')
else
    evalin('base','drawRegionOutline(hROI,1,2,numRaysDop+1);')
end

return
%-EF#1
```

- D. Replace the first line with the delimiter (`%-EF#`) with a real function definition (e.g., `function ROIplot(varargin)`). Replace the closing delimiter (`%-EF#1`) with `end`

```
function ROIplot(varargin) % %-EF#1 ROIplot(varargin)
persistent drawROI

% change the txt for voltage slider
hvltxt = findobj('tag','hvltxt');
hv2txt = findobj('tag','hv2txt');
set(hvltxt,'String','Bmode Voltage');
set(hv2txt,'String','Doppler Voltage');

% drawRegionOutline(WinNum,PDataNum,RegionNum) creates an outline from
% PData(PDataNum).Region(RegionNum) on displayWindow(WinNum) with default color
% white.
if isempty(drawROI)
    drawROI = 1;
    evalin('base','hROI = drawRegionOutline(1,2,numRaysDop+1);')
else
    evalin('base','drawRegionOutline(hROI,1,2,numRaysDop+1);')
end
```

```

return
end % replaced %-EF#1

```

- E. Go back to the line where the EF(1) was defined and replace it with the external function definition

```

EF(1).Function = vsv.seq.function.ExFunctionDef('ROIplot', @ROIplot);

```

- F. Check for external functions that are not defined in the Process structure. Go to the line that defines the external functions (e.g. EF(2) = text2cell('%-EF#2');). Repeat step 2-6 until all external functions are converted to real functions.
- G. If you have external functions that are not defined in the Process structure, they are most likely used in other places in the script. For example, they might be assigned as callbacks to GUI handles. Make sure that all those callback assignments do use **real function handles and are NOT strings**. See moveFocus in the next example

```

.... % somewhere in the script:
% Wrong approach
% set(Resource.DisplayWindow(1).figureHandle,'WindowButtonDownFcn','moveFocus');
% Correct approach:
set(Resource.DisplayWindow(1).figureHandle,'WindowButtonDownFcn', @moveFocus);

...
%-EF#3
function moveFocus(varargin)
%
    freeze = evalin('base','freeze');
    vsExit = evalin('base','vsExit');
    ...
end

```

3.3 Step by step guide UIC

Converting the UIC is very similar to converting the EF.

- A. Start with finding the section in your script that defines the UI controls. This section can easily be identified by the UI(x) definitions.

```

% User specified UI Control Elements

% - Sensitivity Cutoff
UI(1).Control = {'UserB7','Style','VsSlider', ...
                'Label','Sens. Cutoff',...
                'SliderMinMaxVal',[0,1.0,Recon(1).senscutoff], ...
                'SliderStep',[0.025,0.1], ...
                'ValueFormat','%1.3f'};
UI(1).Callback = text2cell('%SensCutoffCallback');

UI(2).Control = {'UserA1','Style','VsSlider', ...
                'Label',['Range (',AxesUnit,')'],...
                'SliderMinMaxVal',MinMaxVal, ...
                'SliderStep',[0.1,0.2], ...

```

```
'ValueFormat', '%3.0f'};
UI(2).Callback = text2cell('%RangeChangeCallback');
```

B. Add an import statement for each UI control in front of the UI control definitions

```
import vsv.seq.uicontrol.VsSliderControl;
```

C. Replace the cell array delimiters of the control with the class name that fits the UIC (e.g., VsSliderControl for 'Style', 'VsSlider') and replace the callback definition with a function handle definition (i.e., @RangeChangeCallback)

```
UI(1).Control = VsSliderControl('LocationCode', 'UserB7', ...
    'Label', 'Sens. Cutoff', ...
    'SliderMinMaxVal', [0, 1.0, Recon(1).senscutoff], ...
    'SliderStep', [0.025, 0.1], ...
    'ValueFormat', '%1.3f', ...
    'Callback', @SensCutoffCallback );

UI(2).Control = VsSliderControl('LocationCode', 'UserA1', ...
    'Label', ['Range (' , AxesUnit, ')'], ...
    'SliderMinMaxVal', MinMaxVal, ...
    'SliderStep', [0.1, 0.2], ...
    'ValueFormat', '%3.0f', ...
    'Callback', @RangeChangeCallback);
```

D. Search for the function definition in your script by the delimiters as defined in the text2cell function. For example if the callback was defined with UI(2).Callback = text2cell('%RangeChangeCallback'); then search for %RangeChangeCallback inside your script

```
%RangeChangeCallback

... code to obtain parameters
P.endDepth = UIValue;
Display = Resource.DisplayWindow(1);

if isfield(Display(1), 'AxesUnits') && ~isempty(Display.AxesUnits)
    if strcmp(Display.AxesUnits, 'mm')
        P.endDepth = UIValue*scaleToWvl;
    end
end

... code to assign the parameters to the system

%RangeChangeCallback
```

E. Replace the delimiters with real function names that have the same name as referenced with the function handle in the callbacks (@function name)

```
% - Range change callback
```

```
function RangeChangeCallback(~, ~, UIValue)
... code to obtain parameters
P.endDepth = UIValue;
Display = Resource.DisplayWindow(1);

if isfield(Display(1), 'AxesUnits') && ~isempty(Display.AxesUnits)
    if strcmp(Display.AxesUnits, 'mm')
        P.endDepth = UIValue*scaleToWvl;           %#ok
    end
end
... code to assign the parameters to the system
end
```

4. Advanced programming and detailed usage

Although backward compatible, we recommend the new approach as the preferred way of defining UIC and EF.

Instead of defining a struct the user can now call functions that will create dedicated UIC or EF objects, which provide more functionality than a pure struct. Nevertheless, the user will be able to use the new objects in a similar fashion as the old structs.

4.1 How to get started: The Package structure

Your best friend is the **TAB** key! Our new software will mostly be organized in packages (see the [Matlab Package](#) documentation for more information). Packages allow us to add more functions without the risk of accidentally overwriting other functions. We hope that we prevent conflicts with software that was written by our users, and thus providing the most flexibility when developing new applications.

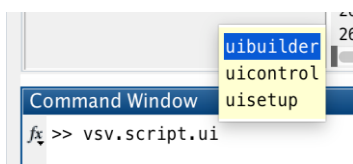


Figure 6 using TAB key after '.' in package structure provides code suggestions

All our new package code starts with **vsv**. Start typing **vsv.seq.ui** then hit the **TAB** key. Matlab will provide a selection of possible items (Figure 6). These items can be sub-packages, functions, or classes. All our classes that create user objects will have upper-case letters. All functions and packages will have lower-case letters.

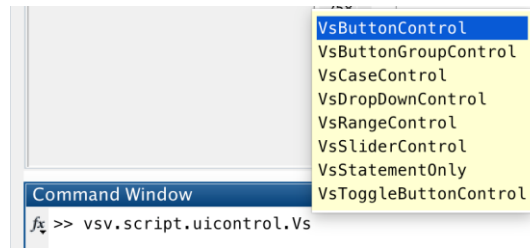


Figure 7 Suggestion of UIC

All new UI controls can be found in [vsv.seq.uicontrol.](#) and all UIC will start with **vs** (Figure 7). The current supported UIC that can be used by the user are:

- `vsv.seq.uicontrol.VsButtonControl`
- `vsv.seq.uicontrol.VsToggleButtonControl`
- `vsv.seq.uicontrol.VsButtonGroupControl`
- `vsv.seq.uicontrol.VsDropDownControl`
- `vsv.seq.uicontrol.VsSliderControl`
- `vsv.seq.uicontrol.VsStatementOnly`

4.2 Getting instant help

Using real objects instead of structs now allows us to provide instant help in the Matlab environment. Simply use the help command and type the following

```
help vsv.seq.uicontrol.VsSliderControl
```

This will produce the following output:

```
VsSliderControl defines a slider control handle that can be added to a GUI

A VsSliderControl inherits from the VsRangeControl, which defines a
range control object in a GUI environment.

to get more information type
>> help vsv.seq.uicontrol.VsRangeControl

To get general information about the UIControl type
>> help vsv.seq.uicontrol.UserUIControl

The UserUIControl is the basic class for all ui controls and provides
information about common attributes such as Callback, and LocationCodes

Usage:
-----
import vsv.seq.uicontrol.VsSliderControl;

vsSlider1 = VsSliderControl( 'LocationCode', 'UserB7', ...
```

```

                                'Label',          'Sens. Cutoff', ...
                                'SliderMinMaxVal', [ 0,1.0,Recon(1).senscutoff],
...
                                'SliderStep',      [0.025,0.1], ...
                                'ValueFormat',      '%1.3f');

% use setter functions to modify some of the fields
vsSlider1.setCallback( @(s,e)disp('Test') );
vsSlider1.setLabel( 'Sensitivity Cutoff' );
vsSlider1.setRange( [0 5] );

Version 1.0 | 2020-01-24
$Author: Dr. Daniel Rohrbach
Copyright 2001-2019 Verasonics, Inc. Verasonics Registered U.S. Patent
and Trademark Office.

Reference page for vsv.seq.uicontrol.VsSliderControl

```

The last line (i.e., [Reference page for vsv.seq.uicontrol.VsSliderControl](#)) is a link that can be clicked. The link will open help information in the Matlab help browser. But you can also just select `vsv.seq.uicontrol.VsSliderControl` and press **F1** or **right click → help** to open the browser-based help.

The browser help looks intimidating at first. The first section provides a general description of the object and how to use it. This is followed by a description of the properties (these are similar to the struct fields as defined in the original approach). The third section provides a list of methods that can be executed on the object (compare Figure 5).

4.3 Usage

4.3.a Easy approach

The easiest way of creating new VsUicontrols is by assigning it to a variable. This will create an object that has all the fields predefined that are necessary to define the object. For example, let's create a slider object:

```
>> slider = vsv.seq.uicontrol.VsSliderControl;
>> slider
```

The output will look like

```

slider =

VsSliderControl with properties:

    SliderStep: [0.0100 0.1000]
      RangeMin: 1
      RangeMax: 100
    ValueFormat: '%3.0f'
    CurrentValue: 1
          Style: 'VsSlider'

```

```

      LocationCode: ''
      Statement: []
      Callback: []
      IsCallbackExecuting: 0
      Label: ''
      EventsEnabled: 0

```

To simplify calling the class creator methods, you can import the entire package:

```

>> import vsv.seq.uicontrol.* % this will import all ui controls
>> toggle = VsToggleButtonControl; % now, you don't need to type vsv.seq.uicontrol
>> toggle

toggle =

VsToggleButtonControl with properties:

      State: 0
      Style: 'VsToggleButton'
      LocationCode: ''
      Statement: []
      Callback: []
      IsCallbackExecuting: 0
      Label: ''
      EventsEnabled: 0

```

4.3.b Use the original definition to create new objects

To make the transition easier for our customers, we also provide the possibility to use the original struct-based definition to create the UI controls. In fact, VSX is using this feature to convert old-struct-based UI definitions into new UIC objects with the added benefits of error checking.

```

>> % UI is a struct as originally defined
>> UI(1).Control = {'UserB7','Style','VsSlider','Label','Sens. Cutoff',...
                  'SliderMinMaxVal',[0,1.0, 0.5],...
                  'SliderStep',[0.025,0.1],'ValueFormat','%1.3f'};
>> slider = vsv.seq.uicontrol.VsSliderControl( UI(1) );
>> slider

slider =

VsSliderControl with properties:

      SliderStep: [0.0250 0.1000]
      RangeMin: 0
      RangeMax: 1
      ValueFormat: '%1.3f'
      CurrentValue: 0.5000
      Style: 'VsSlider'
      LocationCode: 'UserB7'
      Statement: []
      Callback: []
      IsCallbackExecuting: 0
      Label: 'Sens. Cutoff'
      EventsEnabled: 0

```

4.3.c Use value-parameter input

This is a common approach in Matlab to create user objects. The user can pass value-parameter pairs (see Matlab documentation).

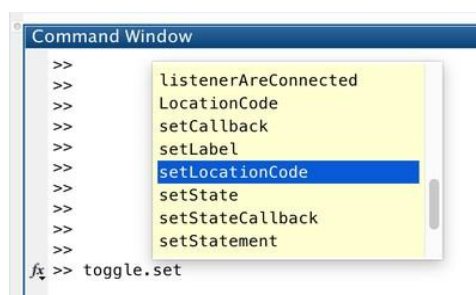
```
slider = vsv.seq.uicontrol.VsSliderControl( 'LocationCode', 'UserB7', ...
                                           'Label','Sens. Cutoff',...
                                           'SliderMinMaxVal',[0,1.0, 0.5],...
                                           'SliderStep',[0.025,0.1], ...
                                           'ValueFormat','%1.3f', ...
                                           'Callback', @(s,e) disp('My Slider
Changed a value') );
```

This approach is very similar to the original approach. However, *the user does not explicitly need to define the 'Style' anymore*, because this property is defined automatically by the class function. Furthermore, to define the location in the Research GUI (e.g., 'UserB7') the user must define the parameter as 'LocationCode'. Before, the first parameter of the cell array defined the location code. With the new approach, the location code can be defined at any location during the creation of the slider control object.

4.4 Properties

Once an object is created some of the properties can be modified by the user. Some properties, however, cannot be modified. This is a feature rather than a limitation, because this way it becomes harder to define invalid user objects. For example, the user is not allowed to change the Style of an object. The style is now defined by the class constructor method and changing the style information would make the object invalid. Or can you imagine a Toggle button that allows you to define a range?

Let's have a look at one examples:



You can easily gain information from your object by using it like a struct using the '.' operator in combination with the **Tab** key on your keyboard.

Let's create a toggle button and let's play around with the properties. Again, use the **Tab** key after the '.' to get suggestions for functions.

```
toggle = vsv.seq.uicontrol.VsToggleButtonControl;
state = toggle.State % use '.' operator to retrieve information about the State
location = toggle.LocationCode % use '.' operator to retrieve information about the
%Location code

state =
    logical
    0

location =
    0x0 empty char array

>> % use setter functions to change state
>> toggle.setState( true )
>> toggle.setLocationCode( 'UserA1' )
>> toggle

toggle =

    VsToggleButtonControl with properties:
        State: 1
        OffLabel: ''
        Style: 'VsToggleButton'
        LocationCode: 'UserA1'
        ParentApplication: []
        Statement: []
        Callback: []
        IsCallbackExecuting: 0
        Label: ''
        EventsEnabled: 0
```

You can also use a more **procedural** approach if you prefer that over the dot notation. However, the procedural approach will not provide the Tab feature for autocompletion.

```
>> setState( toggle, 1 );
>> toggle.State

ans =
    logical
    1
```

Note that the new approach will automatically convert the double input 1 to a logical true.

4.5 Instant Error Checking

A significant advantage of the new approach is the ability to instantly check for errors. The user cannot accidentally define new properties by misspelling anymore:

```
>> toggle.setstate(true)
Unrecognized method, property, or field 'setstate' for class
'vsv.seq.uicontrol.VsToggleButtonControl'.

Did you mean:
>> toggle.setState(true)
```

Furthermore, the user cannot assign invalid parameters

```
>> toggle.setState( 'Accidently a Char' )  
Error using vsv.seq.uicontrol.VsToggleButtonControl/setState (line 71)  
Type of given state is invalid, must be a scalar logical or numeric and not nan
```

Or try to set slider range values with and invalid range that wouldn't make sense:

```
>> vsv.seq.uicontrol.VsSliderControl( 'SliderMinMaxVal', [ 5, 0, 0.5 ] )  
Error using vsv.seq.uicontrol.VsRangeControl/setRange (line 176)  
Min range is larger then Max range
```

Or a value outside the boundaries:

```
>> vsv.seq.uicontrol.VsSliderControl( 'SliderMinMaxVal', [ 0, 1, 5 ] )  
Error using vsv.seq.uicontrol.VsRangeControl/setMinMaxVal (line 202)  
Value is outside of limits
```

Some of the error messages may appear very long. To read them properly, read the first two lines first, because they will provide the information about what went wrong. The following lines are stack trace information that can be helpful for our support team to identify if there are bugs in the software.

When you are trying to execute a script, your script may produce an error message right away before you are trying to execute it in VSX. This seems inconvenient at first, but the advantage is that you will be able to fix it right away before you are trying to run it where it would take much longer to identify the bug.

4.6 Differences to the original approach

As you can see in the examples above, you can still create and define your UIC with the properties as defined in the original approach. However, to provide better features and to improve the logic of our software, we have changed some parameters in the created objects to make usage more explicit and secure.

For example, a slider control now provides methods to set the range. Once created, the slider object defines three parameters for the original argument `SliderMinMaxVal`. The new properties are `'RangeMin'`, `'RangeMax'`, and `'CurrentValue'`.

```
slider = vsv.seq.uicontrol.VsSliderControl( 'Range', [ 5 50] )  
range = slider.getRange()  
cv     = slider.CurrentValue  
  
range =  
     5     50  
cv =
```

Note that the slider control automatically changes the current value to fit into the provided range.

You can directly modify the `RangeMin` and `RangeMax` properties. But make sure to be within the correct range. If the range is valid but `CurrentValue` will run out of bounds, the slider control will set the value to the `RangeMin` or `RangeMax` value

```
>> slider.RangeMin
ans =
     5
>> slider.RangeMax
ans =
    50

>> slider.setRangeMax( 1)
Error using vsv.seq.uicontrol.VsRangeControl/setRangeMax (line 254)
Min range is larger then Max range
```

Each UIC defines a `LocationCode`, which is a string in the form of `'UserA1'`, `'UserA2'`, `'UserB1'`, ..., `'UserB8'`, `'UserC1'`, ..., `'UserC8'`. Depending on the application this code defines the location where to put the custom defined `UIControl`. In the original approach, the user had to define it as the first argument in the `Control` field. In the new approach the user has to explicitly define it as the `'LocationCode'` property. This will allow to place the location code anywhere inside the parameter-value pair input.

4.7 The new Callback function handles

The original approach used a `text2cell` function to parse text segments in the scripts defined by specific delimiters as functions. VSX would then convert these text segments to real m-files and store them as temporary m-files.

The new approach now allows to pass real Matlab supported function handles to your UIC objects. All UIC objects define a `callback` property, which accepts function handles as inputs. A function handle can be created using the at sign:

```
@()myFunction();
```

See Matlab documentation for more help ([How to create function handle](#), [Overview of Function handles](#)) Except for `VsButtonControl`, all function handles for UICs should define three input parameter:

```
@(src, evt, UIValue)myFunction(src, evt, UIValue)
```

If fewer arguments are given, then the function will be called with only the first arguments that fit. Let's say if the user only defines one input argument the function will only be called with the `src` argument.

The `src` is the source graphics object that triggered the event. We provide access to that handle object (`hObject`) to support older scripts. However, we do not recommend modifying GUI components directly. As MathWorks is developing new GUI technologies we may not be able to support the Matlab uicontrols in future releases. The `evt` object provides some means to access the calling application, which also gives access to the other controls. This feature is not fully developed yet.

This document is intended for use by Verasonics customers only.

Do not duplicate or distribute without permission.

Verasonics, Inc.
11335 NE 122nd Way, Suite 100
Kirkland, WA 98034
USA